

Investigação complementar — RequestStatus e filtros da API Pandapé v2 (e v3)

Projeto: EA AUTOMATIC — Fase 5 (INT-1 Pandapé) Data: 2026-07-01 · Ambiente: produção, OAuth2 client_credentials com credencial real Complementa: RELATORIO-INVESTIGACAO-API-PANDAPE-V2-V3.md e INVESTIGACAO-NIVEL-VAGA-PANDAPE.md Sem PII — apenas estrutura técnica, enums, contagens agregadas e códigos HTTP.

Resumo — as 4 respostas

#	Pergunta	Resposta
1	Valores do enum RequestStatus	9 valores (idênticos em v1/v2/v3), documentados abaixo.
2	Existe listagem com filtro por RequestStatus?	NÃO . Nenhum endpoint aceita filtro por RequestStatus — nem documentado, nem "escondido" (variantes não documentadas foram silenciosamente ignoradas ao vivo).
3	Algum status = "candidato aprovado, pronto p/ admissão"?	NÃO . RequestStatus é o ciclo de vida da requisição (aprovação gerencial da vaga), não do candidato. Provado ao vivo: requisição com Status=1 (Approved) e 0 candidatos contratados .
4	Outros parâmetros de filtro não documentados antes?	Sim, 3 filtros úteis (VacancyStatus, VacancyType, includeArchivedRequest) — mas nenhum filtro de data/"mudanças desde" existe (todas as variantes testadas ao vivo foram ignoradas). Um param documentado (Role) não funciona nesta conta.

Q1 — Enum RequestStatus (9 valores)

Tipo **integer**, definido de forma **idêntica em v1, v2 e v3** (components.schemas.RequestStatus). É o ciclo de vida de uma **requisição de cliente** (requisição/pedido de contratação), não do candidato:

Valor	Nome	Significado (ciclo da requisição)
0	PendingApproval	Requisição criada, aguardando aprovação da gestão.
1	Approved	Requisição aprovada pela gestão (autoriza abrir/preencher a vaga). Marca approvedDate.
2	Assigned	Atribuída a um recrutador/responsável para conduzir.
3	InProgress	Recrutamento em andamento (triagem/entrevistas).
4	Completed	Requisição concluída (processo encerrado com sucesso).
5	Rejected	Requisição recusada na aprovação.
6	Archived	Arquivada (fora do fluxo ativo).
7	Draft	Rascunho (ainda não submetida).

8	Cancelled	Cancelada.
---	-----------	------------

Não confundir com **RequestFolderStatus** (enum separado: 0-**Finalist**, 1-**Custom**, 2-**Contracted**, 3-**Discarded**) — **este sim** é o estágio do **candidato** dentro da requisição; 2-Contracted é o equivalente a "contratado". Ver Q3.

Q2 — Listagem com filtro por `RequestStatus`? Não existe

O endpoint de listagem de requisições é `GET /v2/requests`. Parâmetros documentados no swagger: apenas `idVacancy` e `includeArchivedRequest` (boolean). Não há parâmetro `RequestStatus`.

Além disso, testei ao vivo **8 variantes não documentadas** de filtro por status — a API respondeu `200` mas **ignorou todas** (contagem sempre = baseline 13):

Query testada (não documentada)	HTTP	Qtd	Honrado?
baseline <code>GET /v2/requests</code> (sem params)	200	13	—
<code>?RequestStatus=1 / ?requestStatus=1 / ?Status=1 / ?status=1</code>	200	13	■ ignorado
<code>?RequestStatus=4</code> (valor diferente)	200	13	■ ignorado
<code>?IdRequestStatus=1 / ?requestStatusList=1 / ?Statuses=1</code>	200	13	■ ignorado

Observações adicionais confirmadas ao vivo sobre `GET /v2/requests`: - **Retorna array simples e NÃO pagina**: `?Page=1&PageSize=2` e `?Page=2&PageSize=5` → sempre os mesmos 13 (Page/PageSize ignorados). - **includeArchivedRequest=true FUNCIONA**: 13 → 16 (traz as 3 arquivadas). É o único filtro real. - **idVacancy NÃO é um filtro utilizável**: `?idVacancy=847` → **HTTP 400** (e não é obrigatório — a chamada sem ele é que funciona). - **A lista (RequestListItemModel) nem expõe o campo Status** — traz `approvedDate/closedDate` como sinais de ciclo, mas o enum `RequestStatus` só aparece no **detalhe** (`GET /v2/requests/{id}`).

Q3 — Algum status = "candidato aprovado, pronto para admissão"? Não

A premissa não se sustenta: `RequestStatus` descreve a **requisição**, não o candidato.

Prova ao vivo (`GET /v2/requests/583552`): `Status = 1` (Approved), `approvedDate = 2026-02-10`, `closedDate = null`, `contractedCandidatesCount = 0`, `finalist = 0`, `candidatesTotal = 0`. → Ou seja, uma requisição **"Approved"** convive com **zero candidatos contratados**. "Aprovada" significa que a **gestão autorizou a requisição**, não que há candidato pronto para admissão. Nas 13 requisições da conta, **todas** têm `approvedDate` preenchida e `contractedCandidatesCount = 0`.

Onde mora, de fato, "candidato aprovado/contratado" (nenhum é filtrável por uma listagem aberta): - `RequestFolderStatus = 2-Contracted` — estágio do candidato dentro da requisição. - Pasta de etapa da vaga **"Contratados"** (via `GET /v2/matches?IdVacancyFolder=<contratados>`). - Contador `ContractedCandidatesCount` no detalhe da requisição.

Conclusão Q3: não há como "buscar só os prontos para admissão" por um status de requisição — e, mesmo que houvesse esse conceito, **não existe filtro por status na listagem** (Q2). O único caminho de descoberta a nível de candidato continua sendo a listagem de matches por vaga+pasta (já documentado no relatório v2/v3), com o fan-out e as limitações lá descritos.

Q4 — Varredura completa de filtros da v2 (todos os endpoints de listagem)

Testei ao vivo os parâmetros de **todos** os GET de listagem. Resultado consolidado:

Filtros que FUNCIONAM (honrados ao vivo)

Endpoint	Parâmetro	Evidência ao vivo	Novo vs. relatório anterior?
GET /v2/vacancies	VacancyStatus	6822 total → =1 21, =2 907, =3 5894, =7 0	■ agora medido
GET /v2/vacancies	VacancyType	=1 → 16 (filtra)	■ novo
GET /v2/requests	includeArchivedRequest	13 → 16	■ novo
GET /v2/matches	IdVacancy, IdVacancyFolder	já documentado (253 na vaga 847)	—
Paginação	Page, PageSize	honrada nos paginados (clients, headquarters, data-sources, dictionaries, matches, vacancies)	—
Filtros por id-pai	IdClient, IdDatasource, idCategory1, idLocation1/2, idStudy1, idDeficiency1, idMatch, postalCode+limit	filtros hierárquicos padrão	—

VacancyStatus é o achado prático de Q4: dá a contagem de vagas realmente ativas/publicadas = 907 (VacancyStatus=2), muito abaixo do total histórico de 6.822 — número relevante para o dimensionamento de fan-out do spike (Q3 do plano de spike).

Parâmetro documentado que NÃO funciona

Endpoint	Parâmetro	Evidência
GET /v2/company/users	Role	Role=1/2/3 → sempre 61 (= sem filtro). Ignorado nesta conta.

Filtros de DATA / "mudanças desde" — CONFIRMADO que NÃO EXISTEM

Testei ao vivo variantes não documentadas de delta temporal em matches e requests — todas ignoradas (contagem = baseline):

Endpoint	Variantes testadas (ignoradas)	Baseline	Resultado
GET /v2/matches?IdVacancy=18	ModifiedSince, modifiedSince, since, updatedAfter, InsertDateFrom, ModifyDateFrom (=2030-01-01)	253	253 em todas → ignorado
GET /v2/requests	ModifiedSince, CreationDateFrom, since (=2030-01-01)	13	13 em todas → ignorado

→ Não há filtro server-side de "mudanças desde" em lugar nenhum. Isso reconfirma a conclusão do relatório v2/v3: qualquer delta teria de ser feito no cliente (por modifyDate), e o discovery por pull não ganhou nenhum atalho de status/data nesta varredura.

Conclusão

- 1. RequestStatus tem 9 valores (ciclo da requisição), iguais em v1/v2/v3 — documentados acima.
- 2. Não existe filtro por RequestStatus em nenhuma listagem (nem documentado, nem oculto — variantes ignoradas ao vivo). A listagem de requisições nem sequer devolve o status na lista.
- 3. Nenhum status significa "candidato pronto para admissão" — RequestStatus é da requisição, não do candidato (provado: Approved com 0 contratados). O conceito de candidato contratado vive em RequestFolderStatus=Contracted / pasta "Contratados" / ContractedCandidatesCount, não filtráveis por uma busca aberta.

4. **Filtros úteis novos:** `VacancyStatus` (→ **907 vagas ativas/publicadas**), `VacancyType`, `includeArchivedRequest`. **Sem filtro de data/delta** em nenhum endpoint (reconfirmado ao vivo). `Role` em `company/users` está documentado mas **não filtra**.

Impacto na decisão de arquitetura: esta varredura **não abre** nenhum novo caminho de descoberta por *pull* — não há atalho por status de requisição nem por data. O quadro do relatório v2/v3 permanece: o único mecanismo de enumeração de candidatos continua sendo `matches` por vaga+ pasta, com as 4 lacunas já mapeadas. O ganho colateral prático é o número de **vagas ativas (907)** para dimensionar o fan-out.

Anexo — Evidências ao vivo (2026-07-01, credencial real)

Chamada	HTTP	Observação
GET /v2/requests	200	array de 13 (não pagina; todas <code>approvedDate</code> preenchida, <code>contracted=0</code>)
GET /v2/requests?includeArchivedRequest=true	200	16 (13 + 3 arquivadas)
GET /v2/requests?idVacancy=847	400	<code>idVacancy</code> não é filtro utilizável aqui
GET /v2/requests?RequestStatus=1.3.4 (e variantes)	200	13 (ignorado)
GET /v2/requests/583552	200	<code>Status=1</code> (Approved), <code>approvedDate</code> set, <code>contractedCandidatesCount=0</code>
GET /v2/vacancies?VacancyStatus=2	200	907 (ativas/publicadas) — total geral 6822
GET /v2/vacancies?VacancyStatus=1/3/7	200	21 / 5894 / 0
GET /v2/matches?IdVacancy=847&<delta>=1.3.4	200	253 em todas as variantes de data (ignorado)
GET /v2/company/users?Role=1/2/3	200	61 em todas (Role ignorado)

Specs: .../swagger/v{1,2,3}/swagger.json. **Enums:** `RequestStatus`, `RequestFolderStatus`, `VacancyStatus`.

Resultado de investigação para validação. Somente leitura; sem PII; nenhum código alterado.